

CS 331 – Software Engineering Lab

Assignment 8 – Data Access Layer & Testing

Project: InsightFlow

Part A – Data Access Layer (DAL)

1. Introduction to Data Access Layer

The **Data Access Layer (DAL)** is responsible for communication between the Business Logic Layer and the database. It acts as an abstraction layer that hides database implementation details from the rest of the application.

In the InsightFlow system, the Data Access Layer handles:

- User data storage and retrieval
- Report storage and retrieval
- History logging
- Retrieval of stored data
- Communication with the vector database

The DAL ensures that the Business Logic Layer does not directly interact with the database, improving modularity, maintainability, and security.

System Layer Architecture

Presentation Layer (UI)



Business Logic Layer



Data Access Layer



Database

2. Database Design

InsightFlow uses two types of databases:

Database	Purpose
Application Database	Stores users, reports, and history
Vector Database (Qdrant)	Stores embeddings for RAG

3. Database Tables

3.1 Users Table

Stores user authentication information.

```
CREATE TABLE Users (  
  user_id INT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(50),  
  email VARCHAR(100),  
  password VARCHAR(100)  
);
```

3.2 Reports Table

Stores generated reports.

```
CREATE TABLE Reports (  
  report_id INT PRIMARY KEY AUTO_INCREMENT,  
  user_id INT,  
  report_name VARCHAR(100),  
  report_path VARCHAR(200),  
  created_at DATE,  
  FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

3.3 History Table

Stores user activity logs.

```
CREATE TABLE History (  
    history_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    action VARCHAR(200),  
    timestamp DATETIME,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

4. Data Access Layer Code Implementation

The Data Access Layer provides functions to perform database operations such as inserting users, storing reports, and logging history.

Example DAL Code (Python)

```
import sqlite3
```

```
class UserDAL:
```

```
    def __init__(self, db_path="users.db"):  
        self.db_path = db_path  
        self._initialize_db()
```

```
    def _get_connection(self):  
        return sqlite3.connect(self.db_path)
```

```
    def _initialize_db(self):  
        """Create the database and necessary tables."""  
        conn = self._get_connection()  
        cursor = conn.cursor()  
        # Creating a simple 'users' table  
        cursor.execute("""  
            CREATE TABLE IF NOT EXISTS users (  
                id INTEGER PRIMARY KEY AUTOINCREMENT,  
                name TEXT NOT NULL,  
                email TEXT NOT NULL UNIQUE  
            )  
        """)  
        conn.commit()  
        conn.close()
```

```

def create_user(self, name, email):
    """Insert a new user."""
    conn = self._get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO users (name, email)
            VALUES (?, ?)
            """, (name, email))
        conn.commit()
        user_id = cursor.lastrowid
        return user_id
    except sqlite3.IntegrityError:
        # Handle unique constraint violation on email
        return None
    finally:
        conn.close()

def get_user_by_id(self, user_id):
    """Retrieve a user by ID."""
    conn = self._get_connection()
    cursor = conn.cursor()
    cursor.execute('SELECT id, name, email FROM users WHERE id = ?', (user_id,))
    row = cursor.fetchone()
    conn.close()
    if row:
        return {"id": row[0], "name": row[1], "email": row[2]}
    return None

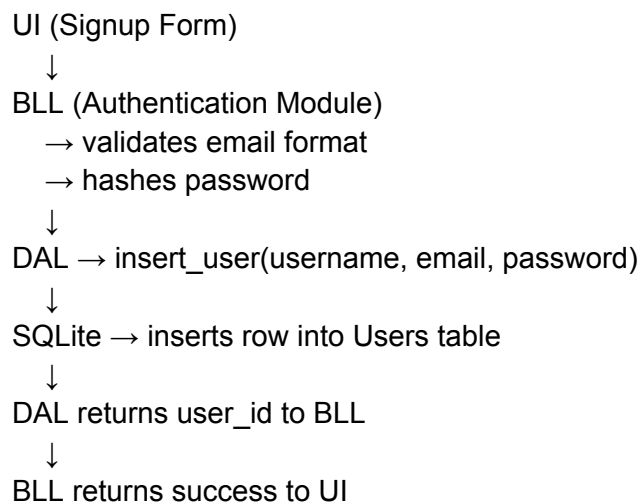
def delete_user(self, user_id):
    """Delete a user by ID."""
    conn = self._get_connection()
    cursor = conn.cursor()
    cursor.execute('DELETE FROM users WHERE id = ?', (user_id,))
    rows_affected = cursor.rowcount
    conn.commit()
    conn.close()
    return rows_affected > 0

```

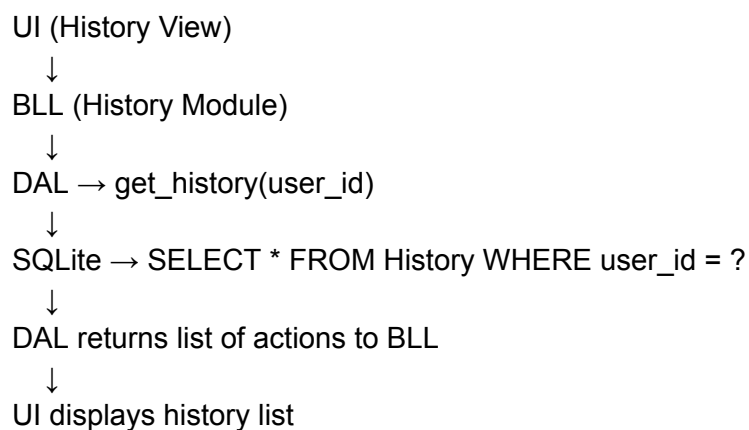
These functions allow the Business Logic Layer to interact with the database without directly writing SQL queries.

How the Functions Are Interacting — Full Flow

Flow 1 — User Signs Up



Flow 2— User Views History



Part B – Testing

5. White Box Testing

White Box Testing is a testing technique where the tester has knowledge of the internal code and logic of the application. The tester checks internal functions, loops, conditions, and logic flow.

White Box Test Cases

Test Case ID	Function Tested	Input	Expected Output
WB1	validate_password()	"123"	Password rejected

WB2	read_csv()	valid.csv	DataFrame created
WB3	embed_text()	"Hello world"	Vector generated
WB4	validate_query()	Empty string	Error message
WB5	log_history()	Upload action	Entry stored
WB6	get_user()	valid email	User record returned
WB7	insert_report()	report data	Report stored
WB8	file_type_check()	.exe file	Upload rejected

White Box Testing ensures that the internal logic of the system works correctly.

6. Black Box Testing

Black Box Testing is a testing technique where the tester evaluates the system without knowing the internal code. The tester focuses on inputs and outputs.

Black Box Test Cases

Test Case ID	User Action	Input	Expected Result
BB1	Signup	Valid user data	Account created
BB2	Login	Valid credentials	Dashboard opens
BB3	Login	Wrong password	Error message
BB4	Upload PDF	Valid PDF	Text extracted
BB5	Upload CSV	Valid CSV	Report generated
BB6	Ask question	Valid query	Answer returned
BB7	Ask question	Empty query	Error message
BB8	View history	Click history	History displayed
BB9	Generate report	CSV uploaded	Report downloaded
BB10	Upload wrong file	.exe file	Upload rejected

Black Box Testing ensures that the system behaves correctly from the user's perspective.

7. Conclusion

The Data Access Layer of InsightFlow manages all database interactions including user data, reports, and history logs. The DAL improves system modularity by separating database operations from business logic. White Box Testing was used to test internal logic and functions, while Black Box Testing was used to test system functionality from the user's perspective. Together, these testing methods ensure the system is reliable and functions correctly.